

Accessing And Updating Array Elements

Generated Aug 17, 2026 7:58 AM

What We'll Learn

By the end of this lesson, students can access and update individual elements in a Java array using index numbers.

Before We Start

If an array stores several student scores, how can we display or change only one specific score?

Let's Understand

Each value stored in an array is called an **element**.

Every array element has an **index number** that identifies its position.

Java array indexes always begin at 0.

For example:

```
int[] scores = {85, 90, 78, 92};
```

The elements and their indexes are:

Index	Element
0	85
1	90
2	78
3	92

The first element is at index 0, not index 1.

Accessing an Array Element

Use the array name followed by the element's index inside square brackets.

```
scores[0]
```

This accesses the first element of the scores array.

```
System.out.println(scores[0]);
```

Output:

```
85
```

To access the third element:

```
System.out.println(scores[2]);
```

Output:

78

Storing an Array Element in a Variable

An array element can be stored in another variable.

```
int firstScore = scores[0];
```

The value of `scores[0]` is copied into `firstScore`.

```
System.out.println(firstScore);
```

Output:

85

Updating an Array Element

Use the assignment operator `=` to replace an element's current value.

```
scores[2] = 88;
```

The value at index 2 changes from 78 to 88.

Before the update:

```
int[] scores = {85, 90, 78, 92};
```

After the update:

```
scores[2] = 88;
```

The array now contains:

85, 90, 88, 92

Assigning Values to an Empty Array

When an array is created using `new`, values can be assigned to its individual elements.

```
String[] names = new String[3];  
  
names[0] = "Angela";  
names[1] = "Joshua";  
names[2] = "Maria";
```

The names array now contains three student names.

Using Array Elements in Expressions

Array elements can be used like regular variables.

```
int[] scores = {85, 90, 78};

int total = scores[0] + scores[1] + scores[2];
double average = total / 3.0;
```

Array elements can also be increased or decreased.

```
scores[0] = scores[0] + 5;
```

If the original value of `scores[0]` is 85, its new value becomes 90.

Finding the Last Valid Index

Use the array's `length` property to determine the number of elements.

```
int[] scores = {85, 90, 78, 92};

System.out.println(scores.length);
```

Output:

```
4
```

Because indexes begin at 0, the last valid index is always:

```
arrayName.length - 1
```

For the `scores` array:

```
scores[scores.length - 1]
```

This accesses the last element, which is 92.

Let's Practice

Practice 1

Study the following array:

```
int[] numbers = {10, 20, 30, 40, 50};
```

Write statements that display:

1. The first element
2. The third element
3. The last element

Practice 2

Update the second element of the following array from 75 to 85.

```
int[] scores = {80, 75, 90};
```

Display the updated value.

Practice 3

Complete the missing index numbers.

```
String[] colors = {"Red", "Green", "Blue"};

System.out.println(colors[___]); // Red
System.out.println(colors[___]); // Green
System.out.println(colors[___]); // Blue
```

Practice 4

Complete the program by assigning the correct values to the array elements.

```
public class Main {
    public static void main(String[] args) {
        String[] names = new String[3];

        // Assign Angela to the first element
        // Assign Joshua to the second element
        // Assign Maria to the third element

        System.out.println(names[0]);
        System.out.println(names[1]);
        System.out.println(names[2]);
    }
}
```

Practice 5

Create an integer array containing three quiz scores. Update the lowest score by adding 5, and then display its new value.